A High-Bandwidth Interleaved Data Memory System

Prathmesh Gabhane
BITS ID: 2025H1400103P
MEL G624 Advanced VLSI Architectures, Assignment 2

1. My understanding and assumptions

The specification leaves a few points open, so before writing the Verilog I set down here how I read the problem and the choices I made.

The system is a data memory for a vector processor. It must let the processor start one 32-bit word access (read or write) every clock cycle, even though a single memory bank needs 6 clock cycles to finish an access. The only way to hide a slow bank is to use several banks and spread the words across them, so that a run of accesses keeps moving from one bank to the next instead of waiting on one. There are eight banks. Each bank is word addressable, holds a 32-bit word per location, and takes a 10-bit address, so each bank has 1024 words and the whole memory has 8192 words, which is 32 KB. The processor gives a byte address; with 8192 words the word address is 13 bits and the byte address is 15 bits, and I take the bits above that to be zero.

The main choice is how words map to banks. I use low-order interleaving, which means consecutive words go to consecutive banks. The bank number is the lowest three bits of the word address and the row inside the bank is the rest:

```
word address = byte_addr[14:2]
bank number  = word_address mod 8 = byte_addr[4:2]
row in bank  = word address / 8   = byte_addr[14:5]
```

This is the mapping that makes a unit-stride stream (word 0, 1, 2, and so on) visit banks 0, 1, 2, and so on in turn. A given bank is then used only once every eight cycles, which is longer than its six-cycle busy time, so it is always free again by the time the stream comes back to it. This is the whole idea of the design and it is what Test 1 shows.

The processor view is big-endian and byte addressable, but it always makes a four-byte word access aligned to a word boundary, so the two low byte-address bits are always zero. Because every access is a whole aligned word, the big-endian byte order inside a word is never actually exercised by the data path; it only means that byte address 4W holds the most significant byte of word W. My model therefore stores and returns whole 32-bit words and ignores the two low address bits.

For the timing of a bank, when a bank latches an access it becomes busy for six cycles and cannot take another access during that time. If the processor tries to use a busy bank the attempt is rejected and the stall signal goes high. I treat stall as an early warning: it is already low during the bank’s last busy cycle, so an access that is held during that cycle is accepted on the edge at which the bank finishes its previous access. This matches the statement that stall is lowered during the clock cycle at the end of which the bank will complete its previous access and be ready to accept a new one. The memory system only rejects the access and raises stall; it does not store the rejected access, so I take it that the processor holds the same access and retries it every cycle until it is accepted. My testbench models this.

The results also come back in a simple way. Because the system accepts at most one access per cycle and every access takes the same fixed six cycles, the accesses finish in the same order they were started, so no reordering logic is needed; the system just reports each completion as it leaves its bank. A write commits to the bank's array when it completes, six cycles after it was accepted, and the write-complete signal is given at that time. The two required test cases are reads, but the model handles writes the same way.

Finally, to make the reads easy to check, each location is seeded so that a read of byte address A returns 0xC0DE0000 with (A divided by 4) in the low bits. So word 0 reads 0xC0DE0000, word 1 reads 0xC0DE0001, and so on, which lets the testbench check both the data and the order.

2. How the memory system works

The system has two kinds of block: eight identical banks, and a thin controller around them.

The bank (mem_bank). Each bank keeps a busy counter. When the controller tells it to take an access, it stores the address, the data and the type, and sets the counter to 6. The counter counts down one per cycle. When it reaches its last step the access completes: for a read the bank loads its 32-bit output register from the array, for a write it writes the array, and it pulses a done signal for one cycle. The bank also gives the controller a combinational "can accept" flag, which is true when the bank is free or is finishing on the coming edge (counter is 0 or 1).

The controller (mem_system). Each cycle it splits the byte address into a bank number and a row (as in Interpretation 1). It looks at the "can accept" flag of the selected bank:

- if the bank can accept, it enables that one bank, which latches the access on the next edge;
- if the bank cannot accept (it is busy), it raises stall and enables no bank, so the access is rejected.

When a bank finishes, the controller passes its done pulse and output register to the processor as read_complete and read_data (or write_complete for a write). Because only one bank finishes in any cycle, this is just a simple selection.

Why 8 banks and 6 cycles. For a unit-stride stream the controller sends access 0 to bank 0, access 1 to bank 1, and so on up to bank 7, then back to bank 0 for access 8. Access 8 reaches bank 0 eight cycles after access 0, but bank 0 only stays busy for 6 cycles, so it is already free. No access ever meets a busy bank, so there are no stalls and the system starts one access every cycle. In short, the design works at full speed as long as there are more banks than the bank access time, and the stride visits different banks.

3. Test setup

The testbench contains a small processor model. Every cycle it presents one read access. If stall is high it holds the same access and tries again next cycle; if stall is low the access is accepted and it moves to the next one. It records when each access is issued and checks each completed read against the expected value and the expected order.

Two test cases are run, as asked:

- Test 1: 16 reads to 16 successive words, byte addresses 0, 4, 8, …, 60 (stride 4 bytes = one word).
- Test 2: 16 reads with a stride of 16 bytes, byte addresses 0, 16, 32, …, 240 (stride 4 words).

4. Results

Test 1 (unit stride)

Every access goes to a different bank in turn, so nothing ever stalls. The 16 reads are issued one per cycle and the results come back one per cycle, in order, after the 6-cycle fill.

```
cyc 1 : ISSUE  read #0  byte_addr=0x000  (bank 0, row 0)
cyc 2 : ISSUE  read #1  byte_addr=0x004  (bank 1, row 0)
cyc 3 : ISSUE  read #2  byte_addr=0x008  (bank 2, row 0)
cyc 4 : ISSUE  read #3  byte_addr=0x00c  (bank 3, row 0)
cyc 5 : ISSUE  read #4  byte_addr=0x010  (bank 4, row 0)
cyc 6 : ISSUE  read #5  byte_addr=0x014  (bank 5, row 0)
cyc 7 : ISSUE  read #6  byte_addr=0x018  (bank 6, row 0)
cyc 8 : ISSUE  read #7  byte_addr=0x01c  (bank 7, row 0)
cyc 8 : COMPLETE read #0 data=0xc0de0000  (ok)
cyc 9 : ISSUE  read #8  byte_addr=0x020  (bank 0, row 1)
cyc 9 : COMPLETE read #1 data=0xc0de0001  (ok)
...
cyc 16 : ISSUE  read #15 byte_addr=0x03c  (bank 7, row 1)
cyc 16 : COMPLETE read #8  data=0xc0de0008  (ok)
cyc 17 : COMPLETE read #9  data=0xc0de0009  (ok)
cyc 23 : COMPLETE read #15 data=0xc0de000f  (ok)
-------------------------------------------------
issued 16 reads over cycles 1..16  (16 cycles)
stalls (rejected attempts) : 0
sustained initiation rate  : 16/16 = 1.00 words/cycle
```

All 16 reads are accepted in 16 back-to-back cycles with no stalls, and every value is correct and in order. This is the full bandwidth the system is designed for: one word per cycle.

Figure 1 shows the simulation waveform for this run.

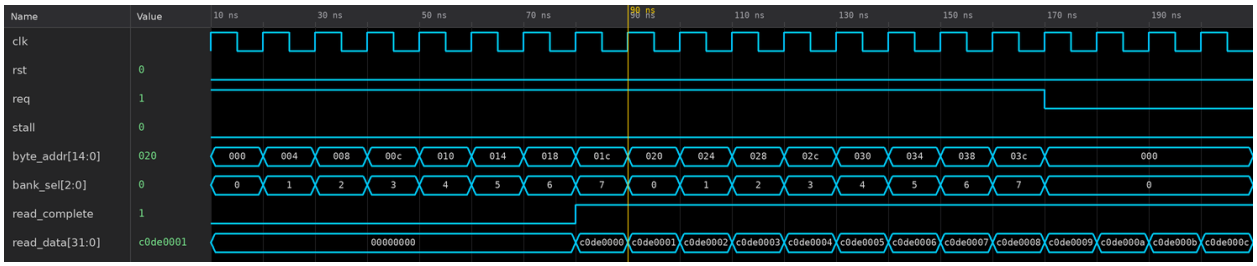


Figure 1. Test 1 (unit stride). req is high every cycle and stall stays low; byte_addr steps through consecutive words and bank_sel cycles through all eight banks, so read_complete asserts once per cycle and read_data steps c0de0000, c0de0001, and so on. The system runs at one word per cycle.

Test 2 (stride of 16 bytes)

A stride of 16 bytes is 4 words. The bank number is (word address mod 8), and the word addresses are 0, 4, 8, 12, 16, …, so the banks used are 0, 4, 0, 4, 0, 4, … Only two of the eight banks are ever used, and each is needed every second access. Since a bank is busy for 6 cycles, the stream cannot keep up and stalls badly.

```
cyc 1 : ISSUE  read #0  byte_addr=0x000  (bank 0, row 0)
cyc 2 : ISSUE  read #1  byte_addr=0x010  (bank 4, row 0)
cyc 3 : STALL  retry byte_addr=0x020  (bank 0 busy)
cyc 4 : STALL  retry byte_addr=0x020  (bank 0 busy)
cyc 5 : STALL  retry byte_addr=0x020  (bank 0 busy)
cyc 6 : STALL  retry byte_addr=0x020  (bank 0 busy)
cyc 7 : ISSUE  read #2  byte_addr=0x020  (bank 0, row 1)
cyc 8 : ISSUE  read #3  byte_addr=0x030  (bank 4, row 1)
cyc 8 : COMPLETE read #0 data=0xc0de0000  (ok)
cyc 9 : COMPLETE read #1 data=0xc0de0004  (ok)
```

```
... (the pattern repeats: two reads issued, then four stall cycles) ...
cyc 43 : ISSUE  read #14 byte_addr=0x0e0  (bank 0, row 7)
cyc 44 : ISSUE  read #15 byte_addr=0x0f0  (bank 4, row 7)
cyc 50 : COMPLETE read #14 data=0xc0de0038  (ok)
cyc 51 : COMPLETE read #15 data=0xc0de003c  (ok)
------------------------------------------------
issued 16 reads over cycles 1..44  (44 cycles)
stalls (rejected attempts) : 28
sustained initiation rate  : 16/44 = 0.36 words/cycle
```

After the first pair, the system settles into a steady pattern of two reads issued, then four stall cycles, i.e. two accesses every six cycles. That is 2/6 = 0.33 words per cycle in steady state; over the whole run it averages 16/44 = 0.36 words per cycle. The data is still correct and in order; only the speed drops.

Figure 2 shows the simulation waveform for this run.

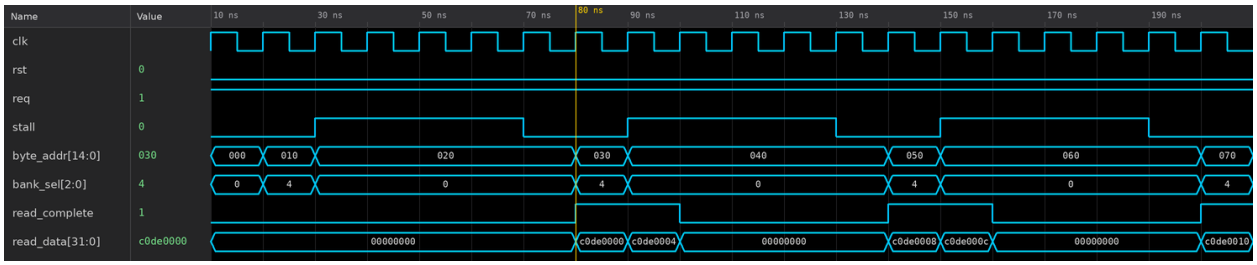


Figure 2. Test 2 (16-byte stride). bank_sel only ever takes the values 0 and 4, so the same two banks are reused; stall is high for four of every six cycles while a busy bank is waited on. The reads still complete in order (read_data c0de0000, c0de0004, …) but far more slowly.

Comparison

	Test 1 (stride 4B)	Test 2 (stride 16B)
Banks used	all 8	only bank 0 and bank 4
Stall cycles	0	28
Cycles to issue 16 reads	16	44
Sustained rate	1.00 word/cycle	0.36 word/cycle

The two runs make the point of the design clear. Interleaving gives the full rate only when consecutive accesses land on different banks. When the stride is a multiple of a factor of the bank count (here 16 bytes = 4 words, and 4 shares the factor 4 with 8), the stream folds onto a few banks and the 6-cycle bank time shows through as stalls. The same memory therefore delivers about three times the bandwidth on Test 1 as on Test 2 purely because of how the addresses map to banks.

5. Conclusion

The memory system meets the goal: with eight banks and low-order interleaving it starts one 32-bit word access per cycle for a unit-stride stream, even though each bank needs 6 cycles per access, because 8 banks is more than the 6-cycle bank time. A busy bank is handled by rejecting the access and raising stall, and the processor holds and retries. Results come out in the order the accesses were made without any extra ordering logic, because every access has the same latency and at most one is accepted per cycle. The simulation confirms correct data and correct order in both test cases, and shows the expected drop from 1.00 to 0.36 words per cycle when a bad stride collapses the stream onto two banks.

The well-commented Verilog for the bank, the memory system and the testbench processor model follows in the appendix.

Appendix: Verilog code

The memory bank and the memory system below are the complete, commented model. The testbench processor model that drives the two test cases follows; it issues one read per cycle and holds the access whenever stall is high.

Memory bank (mem_bank.v)

```
// mem_bank.v  --  one independently addressable memory bank
//
// MEL G624 Advanced VLSI Architectures, Assignment 2
//
// A single word-addressable bank that models a real access latency: once an
// access (read or write) is latched, the bank is BUSY for ACCESS_CYCLES
// (= 6) processor clock cycles and cannot accept another access during that
// time.  The data read out is presented on a built-in 32-bit output register.
//
// Interface (all synchronous to clk):
//   bank_enable : the memory system asserts this to latch a new access on the
//                 next rising edge.  The system only asserts it when the bank
//                 can actually take the access (see accept_ok), so the bank
//                 trusts it.
//   rw_bar      : 1 = read, 0 = write.
//   addr        : 10-bit word (row) address inside this bank.
//   wdata       : 32-bit write data.
//
//   accept_ok   : combinational flag - the bank is free (or frees itself on
//                 the coming edge), so an access presented now would be taken.
//                 The memory system uses this to decide stall.
//   rdata       : output register, holds the data of the last completed read.
//   done        : one-cycle pulse when an access completes (6 cycles after it
//                 was latched).
//   done_rw     : type of the access that just completed (1 read / 0 write).

module mem_bank #(
    parameter integer ADDR_W        = 10,   // 1024 words per bank
    parameter integer DATA_W        = 32,
    parameter integer ACCESS_CYCLES = 6,    // bank busy time in clk cycles
    parameter integer BANK_ID       = 0,    // which bank this is (for init data)
    parameter integer BANK_COUNT    = 8     // total banks (for init data)
)(
    input  wire              clk,
    input  wire              rst,
    input  wire              bank_enable,
    input  wire              rw_bar,
    input  wire [ADDR_W-1:0] addr,
    input  wire [DATA_W-1:0] wdata,
    output wire              accept_ok,
    output reg  [DATA_W-1:0] rdata,
    output reg               done,
    output reg               done_rw
);
    // width of the busy counter (needs to hold 0..ACCESS_CYCLES)
    localparam integer CW = $clog2(ACCESS_CYCLES + 1);

    reg [CW-1:0]     busy;      // cycles left until the bank is free (0 = idle)
    reg [ADDR_W-1:0] a_addr;    // address of the access currently in flight
    reg [DATA_W-1:0] a_wdata;   // write data    "        "        "
    reg              a_rw;      // type          "        "        "

    reg [DATA_W-1:0] mem [0:(1<<ADDR_W)-1];   // the storage array

    // The in-flight access reaches its final (6th) cycle when busy == 1: on
    // this edge it completes, and the bank may immediately take a new access.
    wire completing = (busy == {{(CW-1){1'b0}}, 1'b1});

    // The bank can latch a new access on the coming edge if it is idle (busy
    // == 0) or finishing on that same edge (busy == 1).  Exposed to the
    // system so it can raise stall when busy >= 2.
    assign accept_ok = (busy <= {{(CW-1){1'b0}}, 1'b1});

    // Memory initialisation (simulation model only).
    // The global word address of row r in this bank, under low-order
    // interleaving, is  W = r*BANK_COUNT + BANK_ID.  Each location is seeded
    // with 0xC0DE_0000 | W so a read returns an easily recognised value and
    // the testbench can check both data and completion order.
    integer g;
    initial begin
        for (g = 0; g < (1<<ADDR_W); g = g + 1)
            mem[g] = 32'hC0DE_0000 | (g*BANK_COUNT + BANK_ID);
    end

    always @(posedge clk) begin
```

```
        if (rst) begin
            busy    <= {CW{1'b0}};
            done    <= 1'b0;
            done_rw <= 1'b0;
            rdata   <= {DATA_W{1'b0}};
        end else begin
            // ---- completion of the access that is finishing this edge ------
            if (completing) begin
                done    <= 1'b1;
                done_rw <= a_rw;
                if (a_rw) rdata        <= mem[a_addr];   // read : present data
                else      mem[a_addr]  <= a_wdata;       // write: commit now
            end else begin
                done <= 1'b0;
            end

            // ---- acceptance of a new access (may share the completion edge) -
            if (bank_enable) begin
                a_addr  <= addr;
                a_wdata <= wdata;
                a_rw    <= rw_bar;
                busy    <= ACCESS_CYCLES[CW-1:0];
            end else if (busy != {CW{1'b0}}) begin
                busy <= busy - 1'b1;
            end
        end
    end
endmodule
```

Memory system (mem_system.v)

```
// mem_system.v  --  high-bandwidth interleaved data memory system
//
// MEL G624 Advanced VLSI Architectures, Assignment 2
//
// Eight independent memory banks, each with a 6-cycle access time, arranged
// so the processor can (ideally) start one 32-bit word access every clock
// cycle.  The trick is LOW-ORDER INTERLEAVING: consecutive words live in
// consecutive banks, so a unit-stride stream touches a given bank only once
// every 8 cycles - longer than the 6-cycle bank busy time - and never stalls.
//
// Processor-side interface (big-endian, byte addressable, always a 4-byte
// aligned word access):
//   req           : the processor is attempting an access this cycle.
//   rw_bar        : 1 = read, 0 = write.
//   byte_addr     : byte address of the word (bits [1:0] are 0, aligned).
//   wdata         : 32-bit write data.
//
//   stall         : high when this cycle's attempt hit a busy bank and was
//                   rejected.  The processor must hold the access and retry.
//   read_data     : data of the read that completed this cycle.
//   read_complete : one-cycle pulse, a read finished (read_data valid).
//   write_complete: one-cycle pulse, a write finished.
//
// Address decode (BANKS = 8, 1024 words/bank -> 13-bit word address):
//   word_addr = byte_addr[14:2]
//   bank_sel  = word_addr[2:0]   = byte_addr[4:2]     (low-order interleave)
//   row_addr  = word_addr[12:3]  = byte_addr[14:5]
//
// Completion order: the system accepts at most one access per cycle and every
// access has the same fixed 6-cycle latency, so completions naturally emerge
// in the exact order the accesses were issued - no reorder buffer is needed.

module mem_system #(
    parameter integer BANKS         = 8,
    parameter integer BANK_ADDR_W   = 10,
    parameter integer DATA_W        = 32,
    parameter integer ACCESS_CYCLES = 6,
    // byte address width is derived: 2 offset + log2(BANKS) bank + row bits.
    // Declared here (not as a body localparam) so it can size the port below.
    parameter integer BYTE_ADDR_W   = $clog2(BANKS) + BANK_ADDR_W + 2
)(
    input  wire                     clk,
    input  wire                     rst,
    input  wire                     req,
    input  wire                     rw_bar,
    input  wire [BYTE_ADDR_W-1:0]   byte_addr,
    input  wire [DATA_W-1:0]        wdata,
    output wire                     stall,
    output reg  [DATA_W-1:0]        read_data,
    output reg                      read_complete,
    output reg                      write_complete
);
    localparam integer BANK_SEL_W   = $clog2(BANKS);                // 3
    localparam integer WORD_ADDR_W  = BANK_SEL_W + BANK_ADDR_W;      // 13

    // ---- address decode ------------------------------------------------------
    wire [WORD_ADDR_W-1:0]  word_addr = byte_addr[BYTE_ADDR_W-1:2];
    wire [BANK_SEL_W-1:0]   bank_sel  = word_addr[BANK_SEL_W-1:0];
    wire [BANK_ADDR_W-1:0]  row_addr  = word_addr[WORD_ADDR_W-1:BANK_SEL_W];

    // ---- per-bank signals ----------------------------------------------------
    wire [BANKS-1:0]        b_accept_ok;
```

```
    wire [BANKS-1:0]        b_done;
    wire [BANKS-1:0]        b_done_rw;
    wire [DATA_W-1:0]       b_rdata [0:BANKS-1];
    reg  [BANKS-1:0]        b_enable;

    // One-hot enable: assert only the selected bank, and only if it can take
    // the access.  A rejected attempt (selected bank busy) raises stall.
    integer e;
    always @* begin
        b_enable = {BANKS{1'b0}};
        if (req && b_accept_ok[bank_sel])
            b_enable[bank_sel] = 1'b1;
    end

    assign stall = req & ~b_accept_ok[bank_sel];

    // ---- bank instances ------------------------------------------------
    genvar b;
    generate
        for (b = 0; b < BANKS; b = b + 1) begin : g_bank
            mem_bank #(
                .ADDR_W        (BANK_ADDR_W),
                .DATA_W        (DATA_W),
                .ACCESS_CYCLES (ACCESS_CYCLES),
                .BANK_ID       (b),
                .BANK_COUNT    (BANKS)
            ) u_bank (
                .clk         (clk),
                .rst         (rst),
                .bank_enable (b_enable[b]),
                .rw_bar      (rw_bar),
                .addr        (row_addr),
                .wdata       (wdata),
                .accept_ok   (b_accept_ok[b]),
                .rdata       (b_rdata[b]),
                .done        (b_done[b]),
                .done_rw     (b_done_rw[b])
            );
        end
    endgenerate

    // ---- completion collection -----------------------------------------
    // At most one bank completes per cycle (one accept/cycle, fixed latency),
    // so a simple OR / priority pick is exact and preserves issue order.
    integer i;
    always @* begin
        read_data      = {DATA_W{1'b0}};
        read_complete  = 1'b0;
        write_complete = 1'b0;
        for (i = 0; i < BANKS; i = i + 1) begin
            if (b_done[i]) begin
                if (b_done_rw[i]) begin
                    read_complete = 1'b1;
                    read_data     = b_rdata[i];
                end else begin
                    write_complete = 1'b1;
                end
            end
        end
    end
endmodule
```

Testbench, processor model (tb_mem_system.v)

```
    // Processor stimulus: drive req / byte_addr combinationally from state so
    // they are stable at each rising edge; hold the access while stalled.
    always @* begin
        if (running && issue_idx < 16) begin
            req       = 1'b1;
            rw_bar    = 1'b1;                       // all test accesses are reads
            byte_addr = base + issue_idx*stride;
        end else begin
            req       = 1'b0;
            rw_bar    = 1'b1;
            byte_addr = 15'd0;
        end
        wdata = 32'd0;
    end

    // advance / collect on each rising edge
    always @(posedge clk) begin
        if (!rst && running) begin
            // an attempt that is not stalled is accepted this edge
            if (req && !stall) begin
                issue_cycle[issue_idx] = cycle;
                if (issue_idx == 0) first_issue_cycle = cycle;
                $display("  cyc %0d : ISSUE  read #%0d  byte_addr=0x%03h  (bank %0d, row %0d)",
                         cycle, issue_idx, byte_addr,
                         (byte_addr>>2) & 7, (byte_addr>>5));
                issue_idx = issue_idx + 1;
            end else if (req && stall) begin
                stall_count = stall_count + 1;
                $display("  cyc %0d : STALL  retry byte_addr=0x%03h  (bank %0d busy)",
```

```
                    cycle, byte_addr, (byte_addr>>2) & 7);
        end

        // completions (at most one per cycle)
        if (read_complete) begin
            last_comp_cycle = cycle;
            if (read_data !== expected_data(base + comp_count*stride)) begin
                $display("  cyc %0d : COMPLETE read #%0d data=0x%08h  *** EXPECTED 0x%08h ***",
                         cycle, comp_count, read_data,
                         expected_data(base + comp_count*stride));
                errors = errors + 1;
            end else begin
                $display("  cyc %0d : COMPLETE read #%0d data=0x%08h  (ok)",
                         cycle, comp_count, read_data);
            end
            comp_count = comp_count + 1;
        end
    end
end
```